

monoPix-scout

Browser Extension for Tracking Pixels



Problem

Privacy and Security Concerns with Tracking Pixels

1

What's happening?

A tracking pixel is a tiny, often invisible image (usually 1x1 pixel) that's embedded into emails, websites, or ads to secretly collect data about user behavior.

2

Why does it matter?

They're invisible to users, making them a subtle way to track activity. This can raise privacy concerns, since data is collected without explicit awareness.

3

Who is affected?

Everyday users browsing news, shopping, or reading email (Gmail/Outlook/Yahoo).

Teams in regulated orgs (edu/health/finance) with higher privacy expectations.

Developers & researchers who need to audit pages/inboxes for hidden telemetry.

4

Why do current defenses fall short?

List-only blockers miss novel/first-party or obfuscated pixels; false positives can break pages.

Power-user tools overwhelm with raw request logs and jargon.

Email controls often block remote images but don't explain what was blocked or why, nor surface beacons sent via scripts.

How was this project managed?

github.com/Qrytics/monoPix-scout/tree/main

Qrytics / monoPix-scout

Type to search

Unpin

Watch 0

Fork 0

Starred 1

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

monoPix-scout

Public

main

1 Branch

0 Tags

Go to file

Add file

<> Code

Qrytics

Update content.js

7:46a2 · 6 hours ago

28 Commits

demo-site

Update index.html

last month

extension

Update content.js

6 hours ago

README.md

Improve README formatting for setup instructions

2 weeks ago

README

monoPix-scout — Tracking Pixel Chrome Extension

An image/vision-analysis tool for detecting and visualising pixel-level changes across video frames using a "MonoPix"-inspired model. Discovers and reveals hidden imagery and tracking assets loaded on the webpages you visit. See what sites are pulling in the background — even when content is invisible or masked by CSS. Developed by Qrytics & TechnicalDree. This repository contains the full implementation, a seeded test page with sample data, evaluation scripts, and supporting modules.

Getting Started

About

Main repository for building chrome extension that identifies tracking pixels for user.

Readme

Activity

1 star

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Languages

Name	Last commit message
..	
public	Delete nothing.md
background.js	Update background.js
content.js	Update content.js
devtools.html	Add devtools.html
devtools.js	Add devtools.js
manifest.json	Update manifest.json
options.html	Add options.html
options.js	Add options.js
popup.html	Update popup.html
popup.js	Update popup.js

Name	Last commit message
..	
cameraPic.png	Add test site images
index.html	Update index.html
pixel-1x1.png	Add test site images

6f_0_509#slide=id.g3acfcaa596f_0_509



MonoPix Scout

This site mode: Observe ▾

Refresh

No tracking activity seen yet on this tab.

← → ↻ ⓘ File C:/Users/mario/Downloads/monoPix-scout-main%20(3)/monoPix-scout-main/demo-site/index.html

This demo-site is meant to simulate 3 different tracking methods: Image pixel, Beacon, Network call

Demo page

Visible image:



1×1 pixel:

^^ The 1 by 1 pixel image is here.

MonoPix Scout

This site mode:

`https://example.com/pixel?f=1`

xhr · 1p · fetch · 9:59:05 AM

MEDIUM

`https://example.com/beacon?demo=1`

beacon · 1p · navigator.sendBeacon · 9:59:05 AM

MEDIUM

`file:///C:/Users/mario/Downloads/monoPix-...`

img · 1p · 1x1 tiny (suspiciously) · 9:59:05 AM

MEDIUM

`file:///C:/Users/mario/Downloads/monoPix-...`

img · 1p · 1x1 tiny (suspiciously) · 9:59:05 AM

MEDIUM

page books is:24 (1)

Page_Hook97J5124

```
etch page hooks is:24
```

Page 100 of 100

page hooks.js:11 ⚙️

1184-1193

Approach and Implementation

```
1   This demo-site is meant to simulate 3 different tracking methods: Image_pixel, Beacon, Network_call
2   <!doctype html>
3   <html>
4     <body>
5       <h1>Demo page</h1>
6       <h2>Visible image:</h2>
7       
8       <h2>1x1 pixel:</h2>
9
10      ^^ The 1 by 1 pixel image is here.
11      
12
13      <script>
14        window.addEventListener("load", () => {
15          <!-- Beacon-like call -->
16          if (navigator.sendBeacon) navigator.sendBeacon("https://example.com/beacon?demo=1");
17
18          <!-- HTTP GET Network call -->
19          fetch("https://example.com/pixel?f=1");
20        });
21      </script>
22    </body>
23  </html>
```

```
// Scoring function to determine danger level
✓ function score(sig) {
    let s = 0;
    if (sig.size && sig.size.w * sig.size.h <= 16) s += 2;
    if (sig.thirdParty) s += 2;
    if (/beacon|fetch|xhr/i.test(sig.reason)) s += 2;
    if (/((pixel|track|open|view|beacon|analytics)/i.test(sig.url)) s += 1;
    return s >= 4 ? "high" : (s >= 2 ? "medium" : "low");
}
```

```
27 const _beacon = navigator.sendBeacon.bind(navigator);
28 navigator.sendBeacon = function(url, data){ report("beacon", url, "navigator.sendBeacon"); return _beacon(url, data); };
29 const _fetch = window.fetch.bind(window);
30 window.fetch = function(input, init){ try{ report("xhr", String(input), "fetch"); }catch{} return _fetch(input, init); };
```

```
1  (function () {
2      const _beacon = navigator.sendBeacon && navigator.sendBeacon.bind(navigator);
3      if (_beacon) {
4  ✓    navigator.sendBeacon = function (url, data) {
5          try {
6              window.postMessage(
7                  { source: "monopix", kind: "beacon", url: String(url), reason: "navigator.sendBeacon" },
8                  "*"
9              );
10         } catch (e) {}
11         return _beacon(url, data);
12     };
13 }
14
15 const _fetch = window.fetch && window.fetch.bind(window);
16 if (_fetch) {
17  ✓    window.fetch = function (input, init) {
18        try {
19            window.postMessage(
20                { source: "monopix", kind: "xhr", url: String(input), reason: "fetch" },
21                "*"
22            );
23        } catch (e) {}
24        return _fetch(input, init);
25    };
26 }
27 })();
```

```

47     if (msg?.type === "SOFT_BLOCK" && msg.url) {
48         document.querySelectorAll(`img[src="${msg.url}"]`).forEach((img) => {
49             try {
50                 const c = document.createElement("canvas");
51                 c.width = img.width || 1;
52                 c.height = img.height || 1;
53                 img.src = c.toDataURL("image/png");
54             } catch {}
55         });
56     }
57 });

```

```

1  [
2    { "id": 10001, "priority": 1, "action": { "type": "block" },
3      "condition": { "urlFilter": "*pixel*", "resourceTypes": ["image","xmlhttprequest","sub_frame"] } },
4    { "id": 10002, "priority": 1, "action": { "type": "block" },
5      "condition": { "urlFilter": "*beacon*", "resourceTypes": ["xmlhttprequest","script"] } }
6  ]

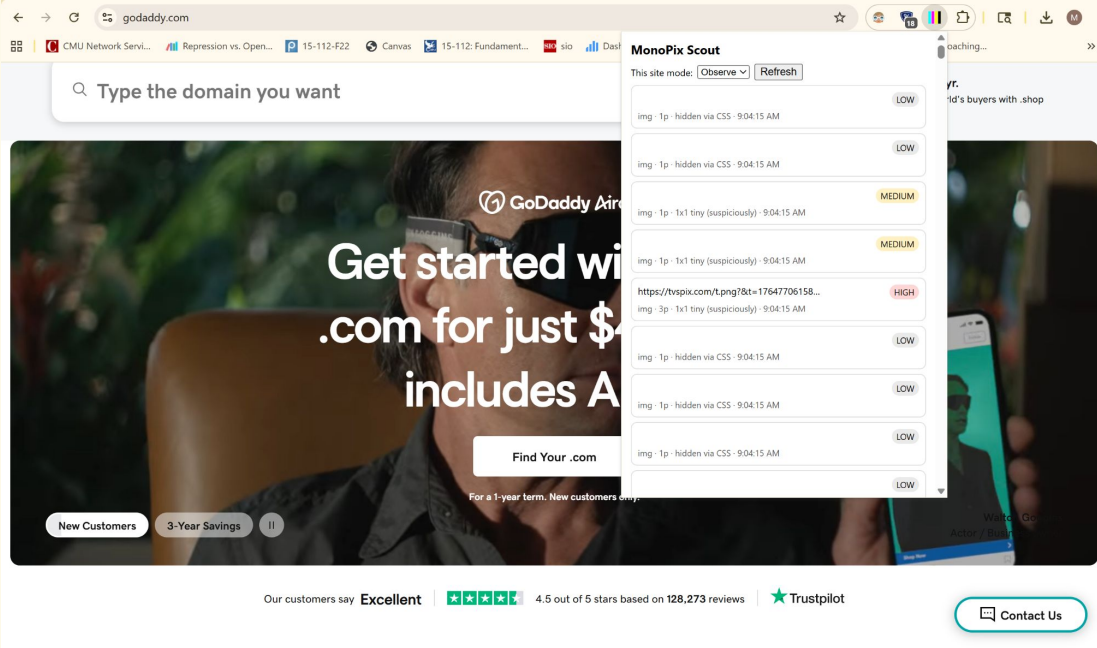
```

Results

MonoPix Scout

This site mode: Observe Refresh

img · 1p · hidden via CSS · 9:03:48 AM	LOW
img · 1p · hidden via CSS · 9:03:48 AM	LOW
img · 1p · 1x1 tiny (suspiciously) · 9:03:48 AM	MEDIUM
img · 1p · 1x1 tiny (suspiciously) · 9:03:48 AM	MEDIUM
https://tvspix.com/t.png?&t=17647706158... img · 3p · 1x1 tiny (suspiciously) · 9:03:48 AM	HIGH
img · 1p · hidden via CSS · 9:03:48 AM	LOW
img · 1p · hidden via CSS · 9:03:48 AM	LOW
img · 1p · hidden via CSS · 9:03:48 AM	LOW
img · 1p · hidden via CSS · 9:03:48 AM	LOW



MonoPix Scout

This site mode:

Strict

Refresh

https://img1.w...nassets/transform...

MEDIUM

img · 3p · hidden · 9:05:55 AM

Observe

Likely

Strict

data:image/gif;base64,R0lGODlhAQABAIAA...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

data:image/gif;base64,R0lGODlhAQABAIAA...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

data:image/gif;base64,R0lGODlhAQABAIAA...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

data:image/gif;base64,R0lGODlhAQABAIAA...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

data:image/gif;base64,R0lGODlhAQABAIAA...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

data:image/gif;base64,R0lGODlhAQABAIAA...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

https://tvpix.com/t.png?&t=17647707318...

HIGH

img · 3p · 1x1 tiny (suspiciously) · 9:05:55 AM

LOW

Did we reach our goals?



Stretch goals

We could add smarter learning, richer email awareness, and deeper visibility into “silent” telemetry

Email-Aware

In Gmail/Outlook, it needs to be able to flag at least 90% of the seeded tracker mails without blocking legitimate content.

DevTools Panel

Rich table of suspect pixels with a URL & size hint, an on-page highlighter, CSV/JSON export.

Adaptive Mode

Automatically notice the trend in appearances of 3rd-party domains; goes through the process: Observe -> Restrict -> Block

Any Questions?

THANK YOU